Terraform-06

1). Watch the **Terraform-06** video.
2). Execute the script shown in the video.
Workspace:

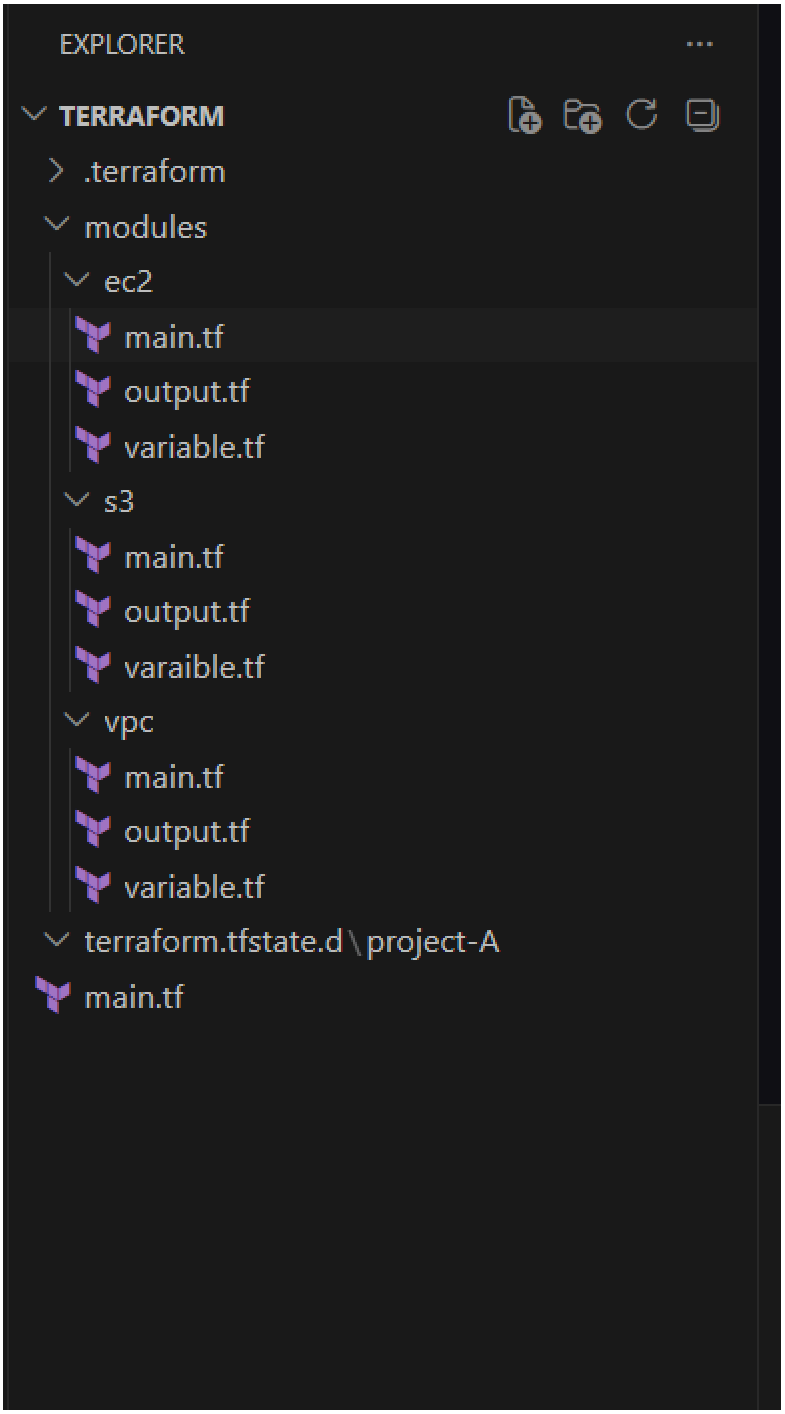
```
 * default

PS C:\Users\DELL\Desktop\Terraform> terraform workspace new project-A
Created and switched to workspace "project-A"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
```

```
PS C:\Users\DELL\Desktop\Terraform> terraform workspace list
  default
* project-A
```

```
PS C:\Users\DELL\Desktop\Terraform> terraform workspace select default
Switched to workspace "default".
PS C:\Users\DELL\Desktop\Terraform> terraform workspace list
* default
  project-A

PS C:\Users\DELL\Desktop\Terraform>
```

```
PS C:\Users\DELL\Desktop\Terraform> cd .\terraform.tfstate.d\
PS C:\Users\DELL\Desktop\Terraform\terraform.tfstate.d> ls


    Directory: C:\Users\DELL\Desktop\Terraform\terraform.tfstate.d


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         6/17/2026  12:10 PM                project-A


PS C:\Users\DELL\Desktop\Terraform\terraform.tfstate.d>
```

3). Provision **EC2, S3, and VPC** using **Terraform modules**.
I have created a terraform project in that created modules, in modules created
vpc, ec2 and s3
In terraform project main.tf and varibale.tf



In EC2 created main.tf

variable.tf .\ | output.tf .\ | main.tf ...\ec2 | variable.tf ...\ec2 | variable.tf

modules > ec2 > main.tf > resource "aws_instance" "server"

```
resource "aws_instance" "server" {
  ami           = var.ami_id
  instance_type = var.instance_type
  subnet_id     = var.subnet_id

  tags = {
    Name = "Terraform-EC2"
  }
}
```

modules > ec2 > variable.tf > variable "subnet_id"

```
variable "ami_id" {}
variable "instance_type" {}
variable "subnet_id" {}
```

In s3 created main.tf

modules > s3 > main.tf > resource "aws_s3_bucket" "bucket"

```
resource "aws_s3_bucket" "bucket" {
  bucket = var.bucket_name
}
```

modules > s3 > varaible.tf > variable "bucket_name"

```
variable "bucket_name" {}
```

In vpc created main.tf

modules > vpc > main.tf > resource "aws_subnet" "public"

```
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr

  tags = {
    Name = "terraform-vpc"
  }
}

resource "aws_subnet" "public" {
  vpc_id     = aws_vpc.main.id
  cidr_block = var.subnet_cidr

  tags = {
    Name = "public-subnet"
  }
}
```

modules > vpc > variable.tf > variable "subnet_cidr"

```
variable "vpc_cidr" {}
variable "subnet_cidr" {}
```

Root main.tf:

```
main.tf > module "s3" > bucket_name
provider "aws" {
  region = "ap-south-1"
}

module "vpc" {
  source      = "./modules/vpc"
  vpc_cidr    = "10.0.0.0/16"
  subnet_cidr = "10.0.1.0/24"
}

module "s3" {
  source      = "./modules/s3"
  bucket_name = "bhargav-terraform-bucket-0291"
}

module "ec2" {
  source          = "./modules/ec2"
  ami_id          = "ami-0f58b397bc5c1f2e8"
  instance_type   = "t3.micro"
  subnet_id       = module.vpc.subnet_id
}
```

Terraform init:

```
SUCCESS: The process with PID 5196 has been terminated.
PS C:\Users\DELL\Desktop\Terraform> terraform init
Initializing modules...
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.50.0...
- Installed hashicorp/aws v6.50.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

Terraform plan:

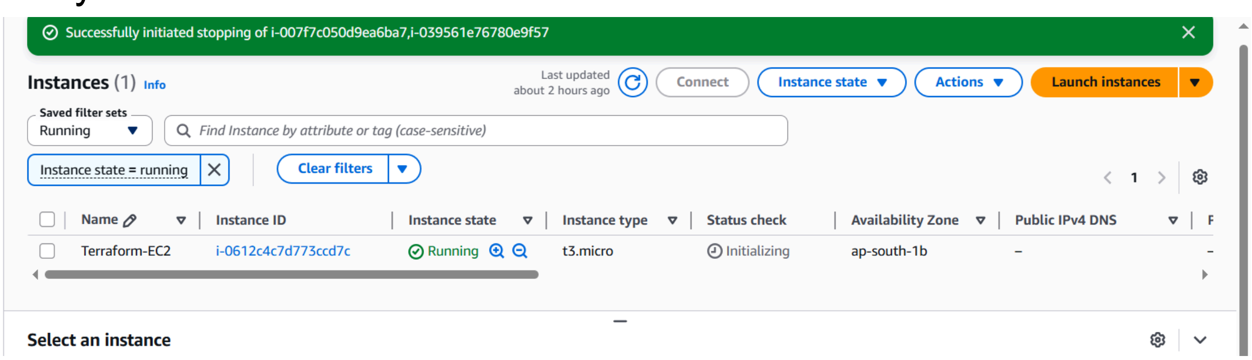
```
PS C:\Users\DELL\Desktop\Terraform> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # module.ec2.aws_instance.server will be created
  + resource "aws_instance" "server" {
      + ami                                  = "ami-0e38835daf6b8a2b9"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
```
Ln 9, Col 2 Spaces: 4 UTF-8 CRLF {} Terraform

Terraform apply:

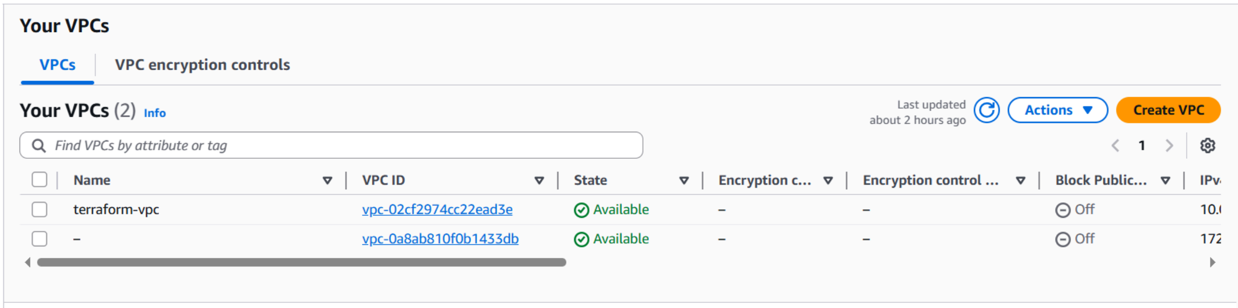
```
PS C:\Users\DELL\Desktop\Terraform> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # module.ec2.aws_instance.server will be created
  + resource "aws_instance" "server" {
      + ami                                  = "ami-0e38835daf6b8a2b9"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
```
Ln 9, Col 2 Spaces: 4 UTF-8 CRLF

```
  Only 'yes' will be accepted to approve.

  Enter a value: yes

module.s3.aws_s3_bucket.bucket: Creating...
module.vpc.aws_vpc.main: Creating...
module.vpc.aws_vpc.main: Creation complete after 2s [id=vpc-02cf2974cc22ead3e]
module.vpc.aws_subnet.public: Creating...
module.s3.aws_s3_bucket.bucket: Creation complete after 2s [id=bhargav-terraform-bucket-0291]
module.vpc.aws_subnet.public: Creation complete after 0s [id=subnet-0eb5aebf084e6e753]
module.ec2.aws_instance.server: Creating...
module.ec2.aws_instance.server: Still creating... [00m10s elapsed]
module.ec2.aws_instance.server: Creation complete after 13s [id=i-0612c4c7d773ccd7c]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
PS C:\Users\DELL\Desktop\Terraform>
```

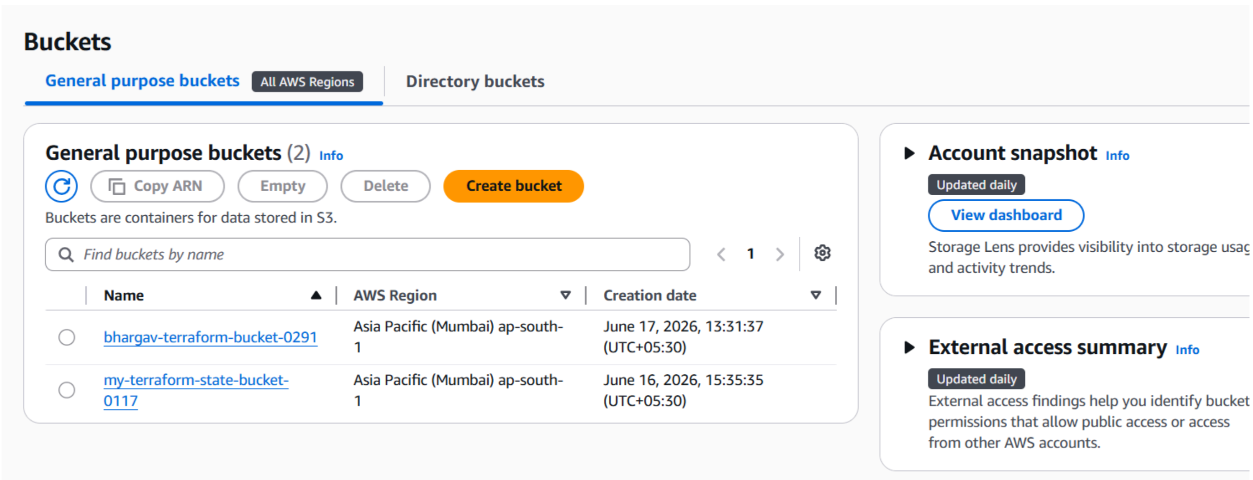
verify:ec2



verify:vpc



Verify: s3



4). Provision **EC2 for 3 different environments** (Dev, Staging, and Prod) using **Terraform workspaces**.

main.tf:

```hcl
provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "server" {
  ami           = "ami-0e38835daf6b8a2b9"
  instance_type = lookup(
    {
      dev     = "t3.micro"
      staging = "t3.micro"
      prod    = "t3.micro"
    },
    terraform.workspace,
    "t3.micro"
  )

  tags = {
    Name        = "${terraform.workspace}-server"
    Environment = terraform.workspace
  }
}
```

outputs.tf

```hcl
output "instance_id" {
  value = aws_instance.server.id
}

output "workspace" {
  value = terraform.workspace
}
```

Terraform init:

```
PS C:\Users\DELL\Desktop\Terraform> terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

Initializing the backend...
Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/aws
- Using previously-installed hashicorp/aws v6.50.0


You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
```

Create Workspaces

Dev:

```
PS C:\Users\DELL\Desktop\Terraform> terraform workspace new dev
Created and switched to workspace "dev"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
```

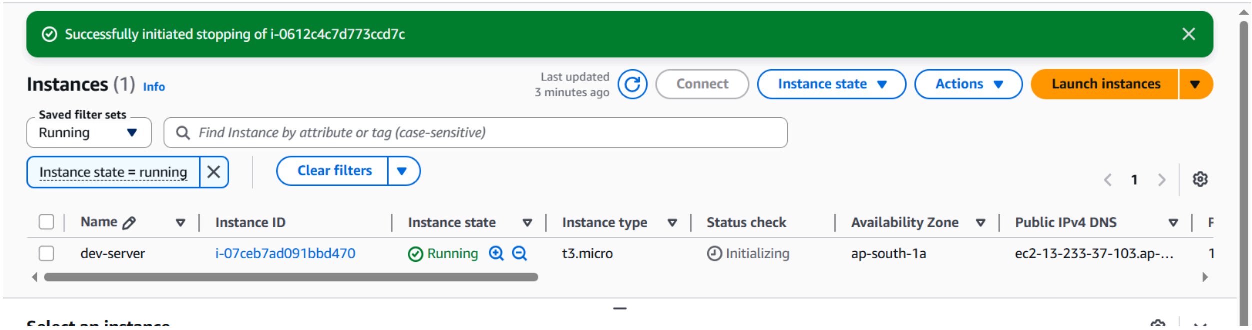
Select workspace:

Terraform apply:

```
PS C:\Users\DELL\Desktop\Terraform> terraform workspace select dev
PS C:\Users\DELL\Desktop\Terraform> terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.server will be created
  + resource "aws_instance" "server" {
      + ami                                  = "ami-0e38835daf6b8a2b9"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
```

```
  + instance_id = (known after apply)
  + workspace   = "dev"
aws_instance.server: Creating...
aws_instance.server: Still creating... [00m10s elapsed]
aws_instance.server: Creation complete after 13s [id=i-07ceb7ad091bbd470]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-07ceb7ad091bbd470"
workspace = "dev"
PS C:\Users\DELL\Desktop\Terraform>
```

verify:



stage:

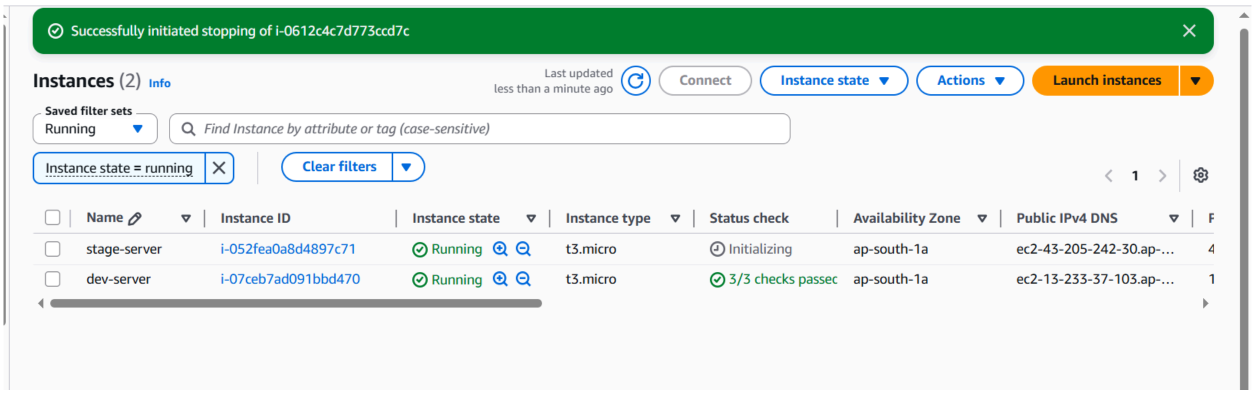
```
PS C:\Users\DELL\Desktop\Terraform> terraform workspace new stage
Created and switched to workspace "stage"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\Users\DELL\Desktop\Terraform> terraform workspace select stage
PS C:\Users\DELL\Desktop\Terraform> terraform workspace list
  default
  dev
  project-A
* stage
```

Terraform apply:

```
PS C:\Users\DELL\Desktop\Terraform> terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.server will be created
  + resource "aws_instance" "server" {
      + ami                                  = "ami-0e38835daf6b8a2b9"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
```

```
Changes to Outputs:
  + instance_id = (known after apply)
  + workspace   = "stage"
aws_instance.server: Creating...
aws_instance.server: Still creating... [00m10s elapsed]
aws_instance.server: Creation complete after 13s [id=i-052fea0a8d4897c71]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-052fea0a8d4897c71"
workspace = "stage"
PS C:\Users\DELL\Desktop\Terraform>
```

verify:



prod:

```
Created and switched to workspace "prod"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\Users\DELL\Desktop\Terraform> terraform workspace select prod
PS C:\Users\DELL\Desktop\Terraform> terraform workspace list
  default
  dev
* prod
  project-A
  stage
```

Terraform apply:

```
PS C:\Users\DELL\Desktop\Terraform> terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.server will be created
  + resource "aws_instance" "server" {
      + ami                                  = "ami-0e38835daf6b8a2b9"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
```

```
Changes to Outputs:
  + instance_id = (known after apply)
  + workspace   = "prod"
aws_instance.server: Creating...
aws_instance.server: Still creating... [00m10s elapsed]
aws_instance.server: Creation complete after 13s [id=i-03e804e87d4518aa5]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-03e804e87d4518aa5"
workspace = "prod"
PS C:\Users\DELL\Desktop\Terraform>
```

verify:

Instances (3) Info

Last updated less than a minute ago | Connect | Instance state | Actions | Launch instances

Saved filter sets: Running

Find Instance by attribute or tag (case-sensitive)

Instance state = running | Clear filters

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
stage-server	i-052fea0a8d4897c71	Running	t3.micro	3/3 checks passec	ap-south-1a	ec2-43-205-242-30.ap-...
prod-server	i-03e804e87d4518aa5	Running	t3.micro	Initializing	ap-south-1a	ec2-13-206-196-26.ap-...
dev-server	i-07ceb7ad091bbd470	Running	t3.micro	3/3 checks passec	ap-south-1a	ec2-13-233-37-103.ap-...